

APIOT: Autonomous Vulnerability Management Across Bare-Metal Industrial OT Networks

Adel ElZemity^{a,*}, Budi Arief^a, Shujun Li^a, George Oikonomou^{b,*}, James Pope^b

^a*University of Kent, Canterbury, United Kingdom*

^b*University of Bristol, Bristol, United Kingdom*

Abstract

Operational technology (OT) devices run safety-critical physical processes, yet their security testing remains manual and expert-intensive. Autonomous security agents could ease this burden, but bare-metal OT devices are difficult targets because they lack shells, filesystems, and named exploit abstractions, requiring operation through protocol fields and observable device state. We present APIOT (Autonomous Purple-teaming for Industrial OT), a large language model (LLM) agent framework that completes a discovery → exploitation → network-level mitigation → verification workflow without human intervention. We evaluate APIOT in 360 runs on vulnerable firmware and simulators for Zephyr RTOS, spanning five LLMs, three topologies, two impairment levels, and prompt, tooling and two ablations. APIOT completed 90% of primary missions (63/70 across protocols and topologies) and 85% of 150 cross-model runs (128/150) — feasible across models, with substantial variation. Named exploit tools (e.g., a canned CoAP option-overflow) were faster, but protocol primitives (the agent builds the fields itself) produced more reusable mitigations that blocked unseen payload variants. Runtime oversight improved reliability: success rose from 27/30 to 29/30 with rule-based guards and 30/30 when a second LLM added advisory steering. To our knowledge, this is the first end-to-end autonomous attack-remediation demonstration on bare-metal industrial devices, establishing controlled feasibility — not zero-day discovery or production-OT transfer.

*Corresponding authors

Email addresses: `ae455@kent.ac.uk` (Adel ElZemity),
`g.oikonomou@bristol.ac.uk` (George Oikonomou)

Keywords: OT security, IoT security, autonomous vulnerability management, LLM agents, bare-metal MCU, industrial protocols

1. Introduction

Operational technology (OT) devices, such as industrial controllers, field sensors, and smart meters, present a growing and under-protected attack surface (Bhamare et al., 2020; Stouffer et al., 2023). Security testing of these systems today is a manual, expert-intensive process: a practitioner must understand the protocol, craft raw payloads, and interpret device responses. This is partly because, unlike IT infrastructure, these devices run bare-metal firmware on resource-constrained microcontrollers (MCUs): at most a minimal real-time operating system rather than a general-purpose one, a rudimentary platform-specific shell typically disabled or stripped in production builds, and only kilobytes rather than megabytes of RAM. They communicate via industrial protocols such as Modbus/TCP (Modbus Organization, 2012) and CoAP (Shelby et al., 2014), which carry device-specific semantics that differ fundamentally from HTTP and SSH. The result is both an engineering bottleneck (scarce expert labour) and a research gap on whether autonomous agents can perform protocol-level exploitation and remediation on bare-metal targets, where device diversity and the scarcity of industrial protocol documentation in large language model (LLM) training corpora leave open whether frontier models can reason at the parser level without fine-tuning.

A first research question therefore concerns *capability*: can autonomous LLM agents complete a protocol-level attack-remediation loop on bare-metal MCU targets? Recent research work has explored whether LLMs can automate penetration testing. Researchers behind PentestGPT (Deng et al., 2024), HackingBuddyGPT (Happe and Cito, 2023), and AutoAttacker (Xu et al., 2024) have shown that LLM agents can navigate Linux privilege escalation and web application attacks with notable autonomy. However, published evaluations overwhelmingly target Linux or web systems. To our knowledge, no prior study has demonstrated an autonomous LLM agent completing a full exploit $\rightarrow$ network-level mitigation $\rightarrow$ verification cycle on bare-metal MCU targets. This remains a difficult research problem because these targets provide no shell or filesystem abstractions, and successful operation requires protocol-level reasoning rather than system-level intuition.

Recent industry efforts corroborate this trend on conventional IT targets. Meta’s CyberSecEval 3 benchmark evaluates LLMs on autonomous offensive cyber operations (Wan et al., 2024), and OpenAI’s o1 system card reports steadily improving performance on offensive capture-the-flag evaluations (OpenAI, 2024). Agentic vulnerability research has also produced real-world findings: Google Project Zero and DeepMind’s Big Sleep agent discovered a previously unknown, exploitable vulnerability in SQLite in 2024 (team, 2024) and, in 2025, pre-empted a second SQLite vulnerability known only to threat actors (CVE-2025-6965) before it could be exploited in the wild (Walker, 2025). Malicious use is following the same curve: OpenAI reports continuing abuse of its models by state-linked and criminal threat actors for cyber operations (OpenAI, 2025), and Anthropic disrupted the first reported AI-orchestrated cyber espionage campaign (GTG-1002), in which an agent executed the bulk of the intrusion lifecycle autonomously (Anthropic, 2025); Anthropic’s 2026 analysis of 832 accounts banned for malicious cyber activity found the share of medium- or high-risk AI-enabled actors rising from 33% to 56% within a single year (Guru et al., 2026). All of these efforts target conventional IT systems—software binaries, web services, and enterprise networks—leaving bare-metal MCU targets and industrial protocols unaddressed.

A second research question concerns *reliability*. LLM agents in autonomous loops are known to exhibit degenerate behaviours: they repeat failed actions, get stuck in subgoal loops, and make inconsistent progress (Liu et al., 2024; Yao et al., 2023). Prior work addresses this primarily through prompt engineering (Wang et al., 2023) or agent-level retries (Shinn et al., 2023). We ask whether an overseer runtime governance layer (Xiang et al., 2025) (rule-based guardrails with optional LLM-based advisory steering) is sufficient to achieve reliable mission completion without fine-tuning, and how large the performance loss is when the overseer is removed.

To address these questions, we designed **APIOT (Autonomous Purpleteaming for Industrial OT)**, an LLM agent framework that completes the full discovery, exploitation, network-level mitigation, and verification cycle on bare-metal OT targets without per-step human intervention. APIOT gives the agent access to low-level protocol primitives rather than named exploit tools, requiring it to operate over protocol fields and observed device behaviour, and includes an overseer runtime governance layer that enforces reliability through rule-based guardrails with optional LLM advisory steering. We evaluated APIOT in controlled experiments totalling 360 runs—70

primary capability runs plus 40 tool-abstraction, 60 runtime-governance, 40 impairment, and 150 model-sensitivity runs (Section 4)—spanning five frontier LLMs (MiniMax M2.5, Gemini 3.1 Pro, Claude Sonnet 4.6, GPT-5.4, and GLM-5), three network topologies, two impairment levels, and two direct ablations (tool abstraction and runtime governance). The evaluated firmware intentionally exposes known protocol-edge-case weaknesses; our goal is to measure autonomous protocol-level operation and network-level mitigation, not to claim zero-day discovery. Our main contributions are:

1. **An empirical demonstration of autonomous attack–remediation on bare-metal MCU targets:** To our knowledge, APIOT is the first end-to-end autonomous LLM agent to complete discovery → exploit → network-level mitigation → verify cycles on bare-metal MCU targets running industrial protocols, without per-step human intervention and across direct and multi-hop OT topologies (Section 5.1). The evaluation uses Zephyr RTOS firmware on ARM Cortex-M MCUs and crash-semantics-equivalent simulators; since the exploits target protocol application layers rather than CPU-specific behaviour, the results are not tied to a specific MCU core. A five-model sensitivity study shows the capability is feasible across frontier LLMs, though with substantial model-to-model variation (Section 5.3).
2. **A tool-abstraction ablation:** We compare protocol-primitive tools against named exploit tools that trigger the same known weaknesses, testing whether the evidence reflects protocol-level construction rather than tool-name selection (Section 5.2). Named tools are faster, but primitives yield observable field construction and mitigations that generalise better to payload variants.
3. **A controlled runtime-governance ablation:** We separate prompt-only operation, deterministic rule guards, and guards plus optional LLM advisory steering (Section 5.4). Deterministic guards account for most of the reliability gain, with advisory steering adding a smaller incremental benefit.
4. **A taxonomy of systematic failures and a deployment envelope:** Across all observed mission failures, we identify four systematic patterns (protocol confusion, stall-and-repeat loops, phase deadlock, and infrastructure failure), enabling targeted countermeasures. In our experiments, the deployment boundary for autonomous CoAP operation appeared in the three-tier Edge–Fog–Cloud topology (Mohan and

Kangasharju, 2016), where traffic traverses multiple hops before reaching the target (Section 5.1).

The rest of this paper is organised as follows. Section 2 provides background and related work. Section 3 presents the APIOT architecture, and Section 4 details the experimental methodology. Section 5 reports empirical results, while Section 6 discusses the implications of these results. Section 7 concludes the paper and outlines directions for future work. Appendix sections provide implementation and configuration details.

2. Background and Related Work

2.1. Bare-Metal MCU Security and Testbeds

Resource-constrained MCUs running real-time operating systems (RTOS) such as Zephyr (Zephyr Project, 2024), FreeRTOS (Real Time Engineers Ltd, 2023), or Contiki-NG (Oikonomou et al., 2022) form the lowest layer of the OT stack. A typical Cortex-M3 device (ARM Limited, 2010) has 64–256 KB of RAM, 256 KB of flash, and runs a single firmware image with no MMU, no process isolation, and no dynamic linking. This means exploitation has no shell to land in and no filesystem to pivot through: success is measured by crashing the device (denial of service) or writing to control registers (actuation attack). We focus on the ARM Cortex-M family (M3 and M4F in our experiments) as one of the most widely deployed 32-bit MCU classes in OT devices and the best supported by mature full-system emulation; ESP32-class devices share the same bare-metal constraints, and because the attacks in this study operate at the protocol application layer rather than on CPU-specific behaviour, the choice of core does not materially affect the results. Among the available RTOS options we use Zephyr: it is open source and vendor-neutral, supports the emulated Cortex-M boards in our testbed, and its Kconfig build system allows a minimal, shell-less image matching the constraints above (details in Appendix A.4).

CoAP (Constrained Application Protocol, RFC 7252; Shelby et al., 2014) is a UDP-based request-response protocol designed for resource-constrained nodes. It supports confirmable (CON) and non-confirmable (NON) messages; option fields encode resource paths and metadata. Malformed option headers (delta/length overflow) can exhaust parser state on devices with no bounds checking (Alghamdi et al., 2013; Arvind and Narayanan, 2019).

Modbus/TCP maps the Modbus application layer over TCP port 502. It uses a Modbus Application Protocol (MBAP) header carrying a length field; devices trusting this field without validation are vulnerable to heap overflow (Lai et al., 2020) when the declared length far exceeds the actual payload.

MQTT (Banks and Gupta, 2014) is a publish–subscribe protocol (TCP/1883) widely deployed for IoT telemetry. Nodes publish messages to named topics on a central broker and subscribe to topics of interest; brokers without enforced access control allow any client to read wildcard topic filters or write to command topics, exposing actuation channels to unauthorised publishers. These three protocols were chosen to cover the two dominant OT interaction styles: request–response access to device state (CoAP, Modbus/TCP) and brokered publish–subscribe telemetry and command (MQTT).

2.2. Autonomous Penetration Testing and Agent Reliability

2.2.1. Autonomous exploitation of conventional IT targets

Deng et al. (2024) used GPT-4 in a reasoning-generation-tool loop to guide human-driven penetration tests, achieving competitive capture the flag (CTF) performance. Happe and Cito (2023) demonstrated fully autonomous Linux privilege escalation with HackingBuddyGPT via tool-calling LLMs. Xu et al. (2024) extended Happe and Cito’s work to network reconnaissance and lateral movement on Linux hosts with AutoAttacker. Happe et al. (2025) systematically evaluated LLMs’ capabilities for privilege escalation across multiple models. Fang et al. (2024) showed that GPT-4 agents could autonomously exploit 87% of one-day Common Vulnerabilities and Exposures (CVEs) given the vulnerability descriptions, extending autonomous exploitation to real-world CVE databases.

All of the above studies evaluated Linux targets with full POSIX semantics. To the best of our knowledge, prior work has not considered bare-metal MCU targets, industrial protocols, or the two-phase red-then-blue workflow (Van Buggenhout, 2024; Scarfone et al., 2008). Our work addresses this research gap.

2.2.2. Agent reliability and runtime governance

Yao et al. (2023) introduced the thought-action-observation loop as a standard LLM agent architecture. Liu et al. (2024) showed that 29 API-based and open-sourced LLMs struggle with long-horizon task completion, exhibiting repetition and inconsistency in complex agentic settings. Recent work on

LLM agent safety and security has proposed various monitoring strategies, including prompt-level instructions and secondary LLM judges (Das et al., 2025; Li and Fung, 2025; Mou et al., 2026; Raptis et al., 2025; Wang et al., 2025). This literature leaves open whether lightweight rule-based guardrails, optionally augmented by a secondary LLM for advisory steering rather than as the primary enforcement mechanism, are sufficient to prevent dominant failure modes.

2.2.3. *Firmware testbeds and discovery on bare-metal targets*

Kim et al. (2020) and Chen et al. (2016) demonstrated that emulation of real firmware can be scaled to thousands of Linux-based router images, establishing dynamic analysis of unmodified firmware as a practical evaluation substrate. Their systems, however, serve automated analysis pipelines rather than interactive agent workloads, and neither emulates bare-metal MCUs. Ghazanfar et al. (2020) proposed IoT-Flock, an open-source framework for generating realistic IoT protocol traffic, which is useful for testing detection systems but models neither bare-metal devices nor adversarial agent workflows. Together, these works provide the emulation and traffic-generation foundations on which we build, while leaving open the need for a testbed that boots bare-metal MCU firmware alongside Linux devices and exposes it to an autonomous agent—the role of IoT Virtual Lab in this paper.

For the discovery stage, the most directly comparable prior work on bare-metal MCUs is firmware fuzzing (automated generation of malformed inputs to discover crashes and memory corruptions in embedded firmware). Clements et al. (2020), Scharnowski et al. (2022), and Feng et al. (2020) emulated MCU firmware and applied coverage-guided fuzzing to find crashes automatically; unlike our framework, this line of work does not address autonomous exploitation, mitigation, or verification stages. APIOT is complementary rather than competing: it operates as a reasoning agent (not a fuzzer), targets the full discovery $\rightarrow$ exploitation $\rightarrow$ network-level mitigation $\rightarrow$ verification cycle, and requires semantic understanding of *why* a payload triggers a parser failure rather than discovering it by exhaustive mutations. IoT Virtual Lab, our QEMU-based emulation testbed, differs in supporting Cortex-M3 bare-metal firmware alongside Linux devices on a shared network, with RESTful API control, multiple industrial IoT (IIoT) topology models, and configurable network realism, properties designed to support autonomous agent evaluation.

Our experiments deliberately use known or seeded protocol-edge weak-

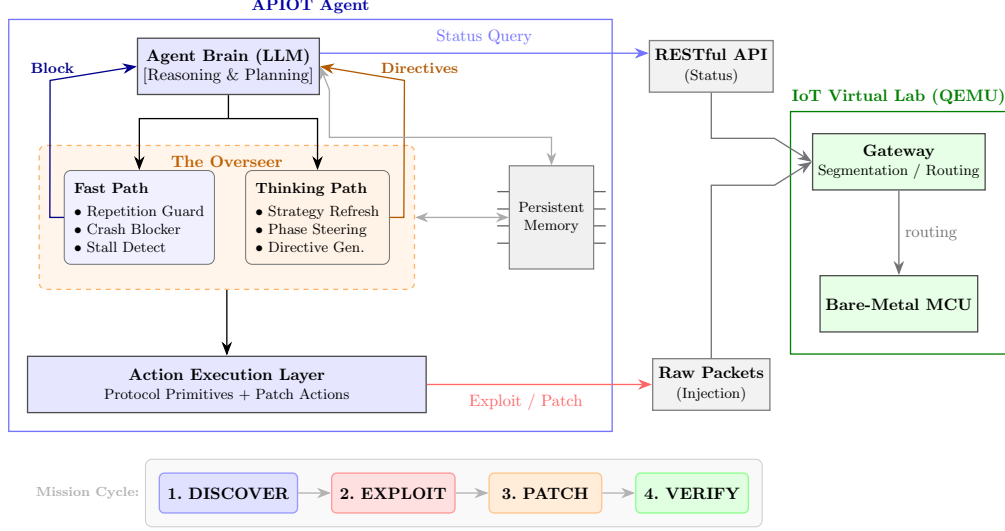


Figure 1: High-level architecture of the APIOT autonomous purple-teaming framework and its interface with the IoT Virtual Lab testbed (see Section 3 for details).

nesses in CoAP, Modbus/TCP, and MQTT. This choice makes the study a controlled evaluation of autonomous protocol-level operation, not a zero-day discovery claim. The question is whether an LLM agent, given low-level protocol and mitigation primitives, can discover the relevant target state, construct payloads, verify impact, derive a network-level mitigation, and validate that mitigation without step-by-step human control. Authenticated and encrypted variants, including CoAP over DTLS, MQTT with enforced broker ACLs/TLS, Modbus security gateways, OPC UA, and DNP3 deployments, are therefore outside the evaluated scope and motivate the secured-protocol experiments discussed in Section 6.

3. System Design

Figure 1 shows the overall architecture of our framework. APIOT is built around an Agent Brain, an LLM reasoning loop backed by a persistent memory store, that drives a four-stage mission cycle of discovery, exploitation, network-level mitigation, and verification. An Overseer sits between the Agent Brain and the Action Execution Layer and enforces reliability through two paths: a rule-based fast path (repetition and crashed-target

guards, phase-transition enforcement) and an optional LLM-based thinking path that provides advisory steering when the fast path detects a stall. IoT Virtual Lab emulates the target network, and APIOT interacts with it solely through low-level raw packets and a read-only RESTful API for status queries; APIOT has no lifecycle control over IoT Virtual Lab, which ensures that measured capability reflects in-mission reasoning rather than orchestration privileges.

3.1. Design Objectives and Scope

The design of APIOT is driven by four requirements. **(R1) End-to-end autonomy**: the agent must complete discovery $\rightarrow$ exploitation $\rightarrow$ network-level mitigation $\rightarrow$ verification within one mission, without per-step operator control. **(R2) Protocol-level reasoning**: the capability test should measure if the agent can infer exploit semantics from protocol structures, not retrieve canned exploit names. **(R3) Runtime reliability**: the architecture must prevent common long-horizon agent failures (repetition, stalled reconnaissance, and invalid phase progression). **(R4) Experimental isolation and reproducibility**: the target environment must be resettable, observable, and strictly separated from agent lifecycle control.

3.2. Core Design Decisions

Protocol primitives over named exploits. The action space exposes low-level protocol primitives rather than pre-packaged exploit labels. This is intended to require explicit selection of protocol fields and parser-facing inputs rather than merely selecting a canned exploit name. Because tool schemas themselves provide affordances, this design choice is evaluated directly through a named-exploit-tool ablation in Section 5.2.

Two-phase purple-team workflow. The mission is structured as Red (discover/exploit/confirm impact) followed by Blue (derive and verify a network-level mitigation). This decomposition aligns the objective with operational purple-team practice (Van Buggenhout, 2024), in which offensive findings are converted directly into validated defensive improvements, and makes failure attribution clearer than a monolithic task objective.

Runtime governance. Overseer places rule-based guardrails on tool-call sequences and mission progression, with optional LLM advisory steering only as a supplementary mechanism. This separates structural safety guarantees from model quality and enables direct ablation of the governance premise.

Strict environment boundary. The agent can observe topology and send packets, but it cannot spawn, reset, or terminate lab devices. This avoids hidden orchestration privileges and ensures measured capability comes from in-mission reasoning and action selection.

3.3. Agent Harness and Prompt Conditions

The harness is part of the method: APIOT implements a continuous thought $\rightarrow$ action $\rightarrow$ observation loop using an OpenAI-compatible chat-completion client and JSON-schema tool calling. At each turn, the model receives the conversation state and the tool schemas, selects one or more tool calls, and the dispatcher executes the corresponding Python implementation. A mission terminates only when the agent emits `TASK_COMPLETE` after both red and blue phases, or `TASK_ABORTED` after an unrecoverable configuration, infrastructure, or budget failure.

Two prompt families are used. Both conditions share the same red-then-blue mission rules, the same mandatory crash and mitigation verification requirements, the same 21 JSON-schema tools, the same overseer settings, the same model temperature, and the same termination rules. The guided prompt additionally provides protocol-specific examples for CoAP option encoding, Modbus MBAP length fields, and MQTT command-topic discovery. The blind prompt removes those protocol-specific exploit hints while preserving the same autonomy, safety, target-knowledge, and tool-use rules, leaving the agent to infer the attack from service observations and tool schemas.

Appendix A.1 records the prompt source identifiers, runner flag for blind mode, tool-schema source file, compaction implementation, and overseer implementation used in the reported experiments. This makes the guided/blind contrast reproducible without relying on prose descriptions alone.

Context management. APIOT estimates context use at approximately four characters per token. When a run exceeds 70% of the selected model’s context window, the compaction layer preserves the system prompt and the latest eight messages, replaces the older middle history with a structured summary, and truncates large tool outputs to 2,000 characters. This preserves recent low-level observations while bounding runaway context growth. The trade-off is explicit: old raw tool outputs may be summarised away, so long-horizon decisions may depend on summary fidelity and persistent memory rather than the full original transcript.

3.4. Protocol-Primitive Tool Interface

The agent does not call a tool named after a vulnerability; it chooses protocol fields and mitigation signatures. Representative red-team primitives include `coap_send`, which takes CoAP message fields plus raw `options_hex` and returns response bytes, timeout status, and round-trip time; `modbus_request`, which takes a function code, PDU bytes, unit ID, and optional MBAP `claimed_length`; and the MQTT pair `mqtt_subscribe`/`mqtt_publish`, which exposes topic filters, topic names, payloads, and message observations. The verification and blue-team side is similarly primitive: `verify_crash` reports alive/crashed state, `iptables_rule` applies protocol/port/payload-match rules, and `verify_patch` replays the prior payload to test whether the mitigation blocks the previously successful attack.

To separate protocol-level operation from tool-name selection, we additionally define a named-exploit-tool control condition. In that condition, the agent receives high-level red-team tools such as `coap_option_overflow` and `modbus_mbap_overflow`, which execute the same known vulnerability triggers without requiring the model to construct CoAP option bytes or MBAP length fields. The blue-team side remains constrained to network-level mitigation and replay verification. This control tests whether faster original-payload triggering also produces reusable mitigation signatures for payload variants that are not exactly the canned trigger.

3.5. Overseer Runtime Governance

The overseer observes each proposed tool call before dispatch. Its fast path is rule-based: it blocks identical calls after three repeats in a 20-call window, prevents attacks against already crashed devices, prevents re-attacking a vulnerability with a verified mitigation, detects stalls after eight turns without progress, and enforces phase transitions when red-team conditions are satisfied. A secondary LLM advisory path can receive a compact state snapshot (phase, recent calls, failures, target status, and untried attacks) and return two or three textual steering directives; it never calls tools directly. We isolate these paths through three execution modes: `off` bypasses both paths, `guards-only` retains deterministic enforcement and heuristic fallbacks but disables secondary-model calls, and `full` enables both guards and advisory steering. This design attributes differences between `off` and `guards-only` to deterministic governance, and differences between `guards-only` and `full` to the incremental advisory path within the evaluated T1 conditions.

3.6. Instantiation for This Study

This conceptual design was instantiated and evaluated on IoT Virtual Lab across CoAP, Modbus/TCP, and MQTT conditions, with topologies T1–T3 (flat star, Purdue Model-segmented, and Edge–Fog–Cloud respectively). Implementation-level parameters (e.g., concrete tool roster, guard thresholds, and device/network configuration) are reported separately in Appendix A.2–Appendix A.4, while the evaluation protocol is defined in Section 4.

4. Experimental Methodology

The evaluation is structured around our two key claims:

- **Claim 1 (capability and robustness):** given protocol-primitive tooling, an autonomous LLM agent can complete the full attack–remediation cycle on bare-metal MCU targets. This is assessed across protocols, topologies, impairment levels, and models, which also establishes the sensitivity of the capability to prompt condition, topology, impairment, and model choice.
- **Claim 2 (oversight as a reliability mechanism):** a runtime governance layer—deterministic guards, optionally augmented by advisory LLM steering—improves mission reliability over prompt-only operation. This is assessed via a three-arm controlled ablation comparing mission reliability and systematic behavioural failures.

Agent decision-making is characterised as a secondary evaluation goal: tool-call distributions and failure modes are logged to describe *how* the agent operates, independently of success/failure outcomes.

4.1. Experimental Conditions

Table 1 summarises the experimental conditions. All runs used a full purple-team mission (both red and blue phases mandatory). Each condition was replicated ten times; the run order was randomised. This design yielded 170 core runs (≈ 85 compute hours; ≈ 30 minutes/run), giving per-cell 95% Wilson confidence intervals with half-width below 15 percentage points near 100% success, which is sufficient to distinguish the ablation and topology effects reported in Section 5. The primary-condition runs fixed a single backbone model (MiniMax M2.5 via OpenRouter) to isolate protocol, topology, impairment, and oversight effects; cross-model robustness was evaluated

Table 1: Experimental conditions. T1 = flat star; T2 = Purdue Model (PERA)-segmented (Williams, 1994); T3 = Edge-Fog-Cloud.

Dimension	Values
Protocol	CoAP (UDP/5683), Modbus/TCP (TCP/502), MQTT (TCP/1883)
Topology	T1, T2, T3
Overseer	Full (guards + advisory), guards only, disabled
Impairment	None; Medium (5% loss, 50 ms latency, 10 ms jitter); Heavy (20% loss, 200 ms latency, 40 ms jitter, HMI traffic)

Table 2: Primary evaluation metrics.

Metric	Definition	Claim
Mission success rate	Percentage of runs reaching TASK_COMPLETE	C1
Exploit confirmation rate	Percentage of attack attempts followed by confirmed impact	C1
Mitigation verification rate	Percentage of network-level mitigations confirmed by verify_patch	C1
Turns to first exploit	Turns before first successful exploit	C1, C2
Turns to completion	Total turns until TASK_COMPLETE	C1, C2
Redundant call rate	Repeated (tool, ip, args) / total calls	C2
Stall events	Turns with no new finding or mitigation	C2
Overseer intervention rate	Steering messages / total turns	C2
Time-to-remediate	Seconds: first exploit $\rightarrow$ verified mitigation	C1
Token use	Estimated maximum context tokens per run where logged	C1, C2

separately in a 150-run model-sensitivity study. The sampling temperature was fixed at the default ($T = 0.7$) for all runs. Heavy impairment included background human-machine interface (HMI) traffic. The T1 flat-star condition is the direct attacker-to-device baseline: for CoAP and Modbus it exposes a single reachable MCU target without an intermediate gateway; T2 and T3 add routing and lateral-movement structure.

Claim 1 evaluation: 70 runs (CoAP/Modbus $\times$ T1/T2/T3 $\times$ 10 repli-

cates + MQTT T1 $\times$ 10). **Tool-abstraction ablation:** 40 runs (CoAP/Modbus $\times$ primitive/named-exploit tools $\times$ 10 replicates). **Claim 2 ablation:** 60 new runs (30 overseer-OFF and 30 guards-only; CoAP/Modbus/MQTT $\times$ T1 $\times$ 10 per arm); the full-overseer arm reuses the corresponding 30 T1 runs from Claim 1. **Robustness:** 40 runs (2 impairment levels $\times$ 2 protocols $\times$ 10). **Model sensitivity:** 150 runs (five models $\times$ three conditions $\times$ ten replicates). **Total: 360 unique runs.** Unless explicitly stated, percentages are reported within their evaluation family rather than pooled across all families.

Six targeted additions were made to support this evaluation, including an overseer-OFF flag, software MCU simulators, a parameterised experiment runner, enriched per-turn logging, analysis scripts, and impairment integration (details in Appendices Appendix A.2–Appendix A.4).

4.2. Metrics

Table 2 lists the primary metrics derived from the tool call log. For the global behavioural characterisation, we define four failure categories coded from session logs: **Protocol confusion** (wrong exploits for device type); **Stall-and-repeat** (identical (tool, ip, args) tuples called more than three times with no progress); **Phase deadlock** (red phase completed, blue phase never initiated despite open findings); and **Infrastructure failure** (device crashed mid-mission causing an inconsistent network state, ending in mission abort).

5. Results

The following results address the two key claims in turn. Sections 5.1, 5.2, and 5.3 evaluate Claim 1: whether protocol-primitive tooling makes bare-metal MCU exploit-remediation tractable in the evaluated setting, whether that effect is distinguishable from named exploit tool selection, and how the capability varies across models, topologies, and network conditions. Section 5.4 evaluates Claim 2: whether the runtime governance layer improves reliability relative to prompt-only operation.

5.1. Autonomous Exploitation and Remediation Capability

Table 3 summarises the topology matrix outcomes for CoAP and Modbus (60 runs: T1/T2/T3 $\times$ 10 replicates each). The results show an all-but-one pattern: APIOT achieved **100%** mission success in every matrix condition

Table 3: Mission success rate by protocol and topology ($n = 10$ per cell) for CoAP and Modbus conditions. Median turns and mission duration are for successful runs only.

Protocol	Topology	Success	Median turns	Median duration
CoAP	T1	10/10 (100%)	11.0	3.0 min
CoAP	T2	10/10 (100%)	30.0	8.5 min
CoAP	T3	3/10 (30%)	18.0	5.0 min
Modbus	T1	10/10 (100%)	24.0	7.5 min
Modbus	T2	10/10 (100%)	23.0	4.5 min
Modbus	T3	10/10 (100%)	27.0	7.5 min
Overall		53/60 (88.3%)	–	–

except CoAP on three-tier topology (T3), where it achieved 3/10 (30%). The additional MQTT baseline condition at T1 achieved 10/10 success; including it yields 63/70 (90.0%; 95% Wilson confidence interval: 80.8–95.1%) overall.

The seven failures occurred exclusively on CoAP T3, the three-tier Edge-Fog-Cloud topology where the MCU target sits behind two forwarding hops on the isolated OT segment. Session log analysis revealed the dominant failure mode as a **multi-target reasoning stall**: after crashing the first device in the OT cell, the agent generated three consecutive empty LLM responses when attempting to reason about lateral movement to the second hop, triggering the empty-response abort guard. The agent initiated the first exploit correctly within 3–4 turns in all failing runs; the breakdown occurs specifically at the multi-hop lateral movement step, not during initial target identification or exploitation. Modbus succeeded in all topologies including T3, where TCP’s stateful connection model provides implicit routing feedback that the stateless CoAP/UDP agent lacks.

Speed of target engagement. Across all 60 tabled runs, the agent initiated its first exploit attempt within 5 tool calls, regardless of topology or protocol. This short recon-to-exploit interval shows that, once given protocol-primitive tools and a reachable target, the agent can move from network state to targeted probing quickly without per-step human direction. Figure 2 shows the full timeline distribution.

Agent decision profile. Across 70 Claim 1 runs (60 CoAP/Modbus + 10 MQTT), the agent made 1,660 tool calls in total (mean 23.7 per run). Figure 3 shows the per-condition tool-call distribution for CoAP and Modbus;

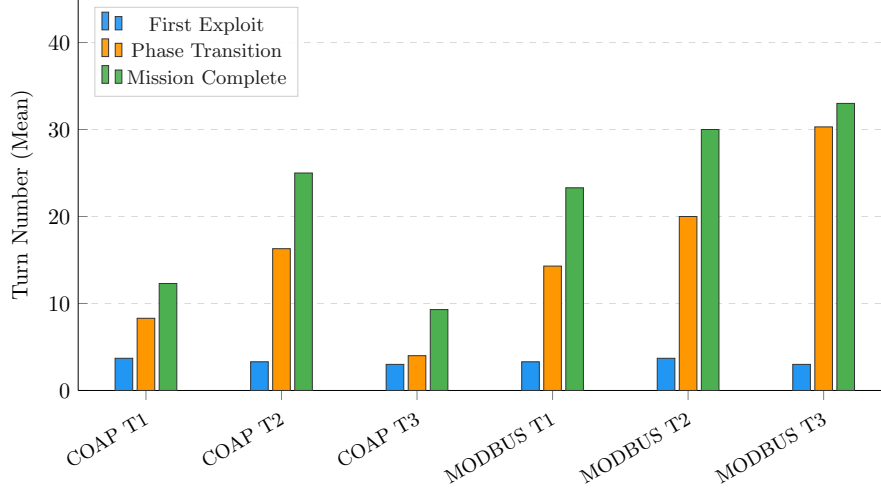


Figure 2: Mean turn counts to three milestones: the first exploit attempt, the red-to-blue phase transition (attack complete, defence begins), and the mission completion. The recon-to-exploit interval (turns 3–4) is consistently short, and most of the mission is spent in the exploit-and-verify loop rather than the blue-team phase.

MQTT’s multi-step attack chain produced a distinct profile discussed in Section 5.4.

Exploit calls account for 27.0–51.0% of all calls across conditions (higher in Modbus, where target identification is faster). **Recon calls** account for 25.0–42.0%, with CoAP T3 showing the highest fraction (32.0%), consistent with the extra routing hops requiring more topology discovery. **Blue-team calls** account for 5.0–19.0%, a notably small fraction reflecting that remediation is concentrated in a short phase after exploit success.

Failure mode. The seven mission failures (CoAP T3) reflect a lateral-movement planning failure mode specific to stateless UDP protocols in multi-hop topologies, not a target-identification or exploit-generation failure. Zero runs exhibited the “zero exploits attempted” or “exploit succeeded, verify failed” failure modes, confirming that the agent’s protocol-specific exploit logic is reliable once it engages a target. All 63/70 successful runs also completed the blue-team phase, with the mitigation verification step confirmed in every case.

Representative attack-remediation traces. Successful traces illustrate the level at which the agent operates: it constructs protocol fields and

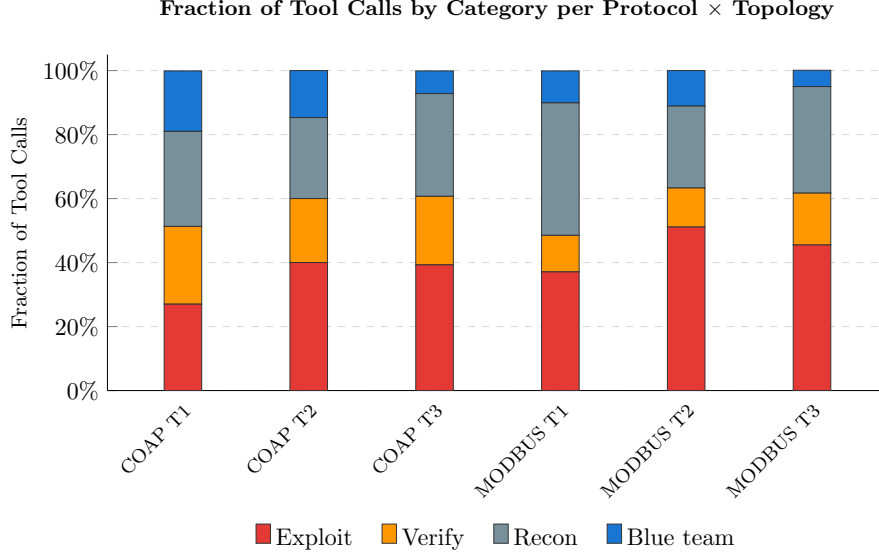


Figure 3: Fraction of tool calls by category (exploit, verify, recon, blue team) per protocol $\times$ topology condition. Exploit and recon together dominate in all conditions; blue-team calls are consistently the smallest category (5.0–19.0%).

network-level mitigation signatures, rather than selecting a pre-named exploit and mitigation pair.

- **CoAP.** The agent sent a normal GET (40011234), then used `coap_send` with malformed option bytes DD FF FF. The target timed out; `verify_crash` confirmed that the CoAP port was unresponsive. The mitigation was an `iptables` hex-string drop for DD FF FF on UDP/5683, followed by replay verification with `verify_patch`.
- **Modbus.** The agent started with an FC3 read probe, then tested anomalous MBAP `claimed_length` values such as 0x0800 and 0x1000, along with FC5/FC6 write probes. The observed effect was a connection reset or timeout followed by crash/alive verification. The mitigation used `u32` function-code filters and TCP/502 signature rules, followed by replay verification.
- **MQTT.** The agent subscribed to #, discovered `iot/sensors/*`, and published `{"action": "shutdown"}` to `iot/commands/shutdown`. A

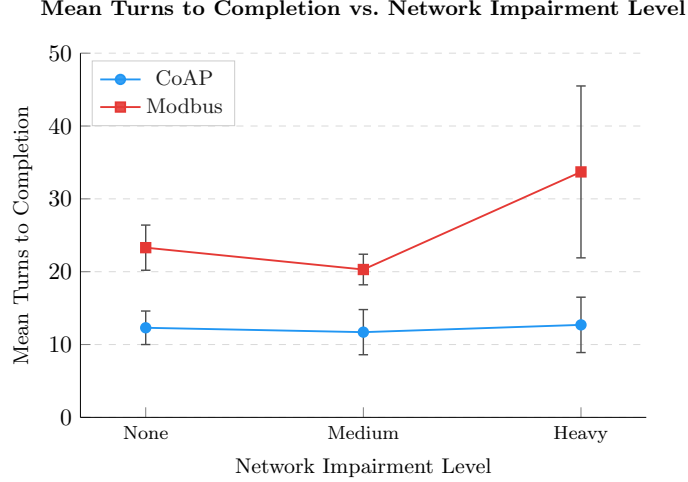


Figure 4: Mean turns to mission completion vs. network impairment level (all runs successful at 100%). CoAP turn count is stable across all conditions (≈ 12 turns); Modbus increases from 23 turns at baseline to 34 turns under heavy impairment (20% loss, 200 ms latency), indicating efficiency degradation without mission failure.

subsequent subscription observed zero telemetry messages, indicating that the command had taken effect. The mitigation used TCP/1883 string-match rules for shutdown/reboot command payloads or broker ACL hardening.

5.2. Protocol Primitives Versus Named Exploit Tools

To test whether the core capability is more than selecting a pre-labelled exploit tool, we compare the protocol-primitive interface against a named-exploit-tool control on the direct T1 baseline. The control exposes high-level tools that trigger the known CoAP option-overflow and Modbus MBAP-length weaknesses directly, while preserving the same mission lifecycle, target discovery, overseer settings, network-level mitigation tools, and replay-verification requirement.

The ablation is not intended to show that primitives are faster or more successful on the easiest known-trigger setting. Named exploit tools reached impact in fewer turns because they collapse protocol construction into one action. The stronger test is whether the primitive interface produces evidence of protocol-level operation: explicit construction of option bytes or

Table 4: Tool-abstraction ablation on T1 conditions. “Variant blocked” measures whether the final network-level mitigation also blocks a held-out payload variant rather than only the original exploit replay.

Protocol	Tool interface	Mission success	Median turns	Field construction	Variant blocked
CoAP	Primitive	10/10	11.0	10/10	9/10
CoAP	Named exploit	10/10	6.0	0/10	3/10
Modbus	Primitive	10/10	24.0	10/10	9/10
Modbus	Named exploit	10/10	8.0	0/10	4/10

MBAP length fields, derivation of protocol-aware network-level mitigation signatures, and blocking of held-out payload variants. The observed pattern is therefore that named tools are faster on original payloads, but primitives produce higher field-construction and variant-blocking rates.

Remediation semantics. A network-level mitigation in this study is a verified traffic-control rule, not a firmware source-code repair. The agent derives a traffic signature from the crash-inducing payload, applies an iptables/ACL-style rule, and replays the same payload after the target watchdog/reset interval. In the retained remediation artifacts, the applied mitigations are primarily hex-string filters (84 rules), protocol/port drops (80 rules), and Modbus u32/function-code filters (15 rules). This captures a realistic emergency containment action for OT networks, but it does not replace vendor firmware repair or production change-control validation.

Token usage. Token accounting was added during the logging-enrichment phase, so it is available for a subset of retained run databases rather than all evaluated runs. Among 61 runs with positive per-turn token logs, the estimated maximum context use per run ranged from 5,980 to 118,656 tokens; the quartiles were 10,053, 14,767, and 20,872 tokens, with mean 17,705. These values are approximate (character-count based) and should be interpreted as operational-cost indicators rather than provider billing records. Three model-sensitivity runs hit explicit cumulative token-budget guards, which prevented runaway cost in long MQTT/blind failures.

5.3. Capability Across Topologies, Impairments, and Models

Figure 4 characterises the impairment aspect of the autonomous OT agent capability envelope.

Topology. Modbus/TCP achieved 100% success across all three topologies, including T3. CoAP/UDP succeeded at 100% on T1 and T2, but degraded to 30% on T3 via the lateral movement stall and TCP/UDP routing asymmetry described in Section 5.1. This establishes three-tier isolated OT cell topology as the current lateral-movement planning boundary for autonomous CoAP deployment, not a network transport limitation.

Network impairment. Both protocols proved robust across all tested impairment levels: CoAP and Modbus each achieved 100% mission success at medium (5% loss, 50 ms latency, 10 ms jitter) and heavy (20% loss, 200 ms latency, 40 ms jitter, HMI background traffic) conditions. CoAP’s UDP-based retransmission handles packet loss without connection state; Modbus/TCP tolerates the latency and jitter introduced without requiring connection re-establishment in these experiments. Network impairment is therefore not a deployment boundary within the tested parameter range.

Model sensitivity tests whether the capability demonstrated in Section 5.1 is tied to one LLM choice. We conducted 150 experiment runs comparing five frontier LLMs (MiniMax M2.5, Gemini 3.1 Pro, Claude Sonnet 4.6, GPT-5.4, GLM-5) across three conditions: guided CoAP T1 (*easy*), multi-step guided MQTT T1 (*hard*), and CoAP T1 without protocol hints (*blind*). Figure 5 summarises the results, while Table 5 shows the efficiency metrics. The results support cross-model feasibility in the easy condition, but also show that harder multi-step tasks expose large model-specific differences.

Easy condition (CoAP T1, guided). All five models achieved 100% success. Efficiency diverges: Gemini 3.1 Pro is fastest (7.7 turns), followed by MiniMax (12.3), GPT-5.4 (13.3), GLM-5 (15.0), and Claude Sonnet 4.6 (16.0). Gemini’s higher redundant-call rate (0.48) versus Claude (0.31) and GPT-5.4 (0.32) suggests it issued more exploratory recon calls before committing to exploitation. This easy condition shows that all tested models can complete the simplest guided protocol-primitive task.

Hard condition (MQTT T1, multi-step guided). This condition is the strongest capability discriminator. GLM-5 and MiniMax M2.5 both achieved 100% success (mean 23.3 and 21.0 turns, respectively); Gemini and Claude each achieved 70% (7/10); and GPT-5.4 achieved 0% (0/10). The GPT-5.4 failures are *task-abandonment* events: all ten runs terminated within 2–3 recon calls (mission completion signalled without issuing any MQTT tool call), despite explicit multi-step guidance in the system prompt. This is a qualitatively distinct failure class from stall-and-repeat: the model exited the mission prematurely rather than engaging the attack chain. The

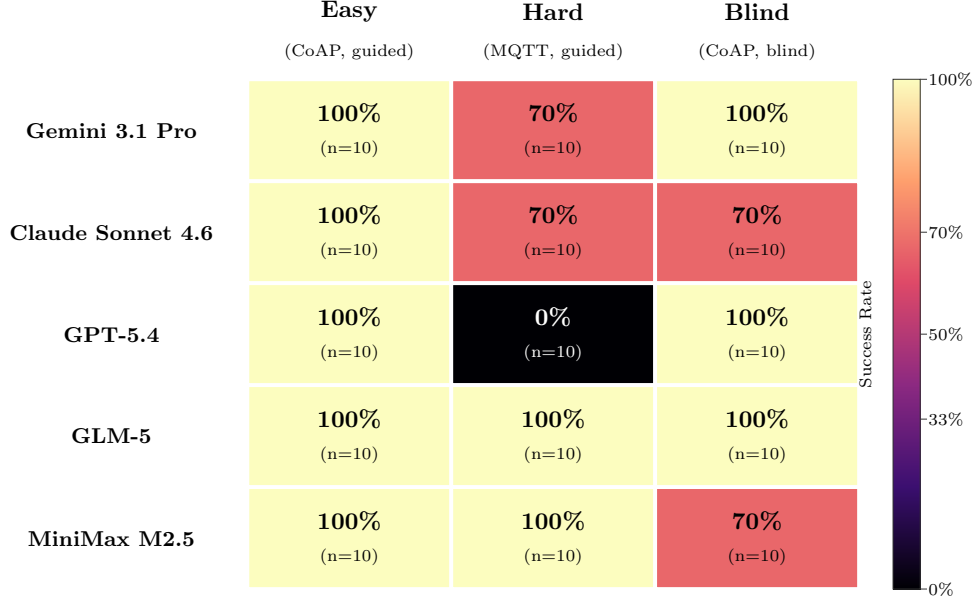


Figure 5: Model sensitivity in mission success rate by model and condition ($n = 10$ per cell). All models achieve 100% on easy (CoAP, guided); the hard condition (MQTT, multi-step) separates models, with GPT-5.4 failing all ten runs; the blind condition (CoAP, no protocol hints) shows Claude and MiniMax M2.5 as models below 100%.

Gemini and Claude failures are genuine execution failures: three runs each stalled in the MQTT topic-discovery phase, identical to the overseer-OFF pattern in Section 5.4. Claude’s successful hard runs required substantially more turns (31.0 vs Gemini’s 14.5 and MiniMax’s 21.0), reflecting its more thorough state exploration before committing to each MQTT operation.

Blind condition (CoAP T1, no protocol hints). Gemini, GPT-5.4, and GLM-5 all achieved 100%; Claude and MiniMax M2.5 each achieved 70% (7/10). The Claude failures are reasoning dead-ends in the protocol-discovery phase: the model issued repeated target enumeration calls without forming a hypothesis about the CoAP option-overflow attack, eventually triggering the repetition guard. The MiniMax M2.5 blind failures are qualitatively similar: the model aborted after 4 turns without attempting exploitation. The three models achieving 100% blind success show that some frontier models can infer the CoAP attack path without prompt-level protocol hints, but the failures

Table 5: Model sensitivity by condition. Turns and redundancy are means over successful runs; dashes denote a cell with no successful runs.

Model	Condition	Success	Turns	Redundancy
Gemini 3.1 Pro	Easy	10/10	7.7	0.48
	Hard	7/10	14.5	0.29
	Blind	10/10	10.3	0.43
Claude Sonnet 4.6	Easy	10/10	16.0	0.31
	Hard	7/10	31.0	0.27
	Blind	7/10	15.5	0.36
GPT-5.4	Easy	10/10	13.3	0.32
	Hard	0/10	–	–
	Blind	10/10	10.3	0.34
GLM-5	Easy	10/10	15.0	0.41
	Hard	10/10	23.3	0.21
	Blind	10/10	13.3	0.45
MiniMax M2.5	Easy	10/10	12.3	0.31
	Hard	10/10	21.0	0.25
	Blind	7/10	9.0	0.35

show that this is not uniform.

Summary. The easy condition was universally solved. The hard (MQTT) condition distinguishes models: GLM-5 and MiniMax M2.5 are the most capable (100%), and GPT-5.4 the least (0%, task abandonment). The blind condition showed that CoAP first-principles reasoning is accessible to several frontier models; Claude and MiniMax M2.5 showed the greatest sensitivity to the absence of protocol hints (both 70%). Across all 150 model-sensitivity runs, the overall success rate is 85.3% (128/150; 95% Wilson confidence interval: 78.8–90.1%), and aggregate success rates across the five models showed a standard deviation of 12.6 percentage points. These dispersion statistics support feasibility beyond one model while making model-to-model variability explicit. The non-monotonic GPT-5.4 pattern (100% easy, 0% hard, 100% blind) should not be read as inconsistent capability: the hard-condition failures were task abandonment before any exploit attempt—an engagement failure, not failed exploitation—whereas the blind condition tests protocol inference, which the model handled. Per-condition results are therefore behavioural profiles rather than a single capability ranking.

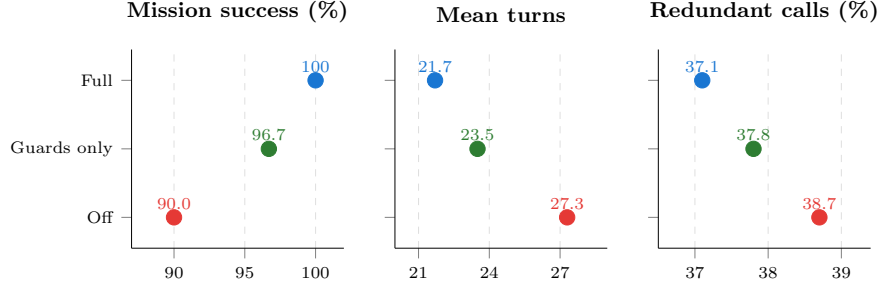


Figure 6: Three-arm runtime-governance comparison on T1. Deterministic guards recover most of the mission-success and mean-turn improvement relative to no oversight; advisory steering provides a further incremental gain. Redundant-call rates remain similar, indicating that governance bounds the consequences of repetition rather than removing the underlying tendency.

Table 6: Three-arm runtime-governance ablation on T1. Mean turns are calculated over successful runs.

Condition	Rule guards	LLM advisory	Mission success	Mean turns
No overseer	No	No	27/30 (90.0%)	27.3
Guards only	Yes	No	29/30 (96.7%)	23.5
Full overseer	Yes	Yes	30/30 (100%)	21.7

5.4. Runtime Guards Explain Most of the Oversight Gain

Figure 6 and Tables 6 and 7 compare three conditions with 30 T1 runs each (10 CoAP + 10 Modbus + 10 MQTT (Banks and Gupta, 2014) per condition).

The three-arm comparison separates the mechanisms behind the module-level effect. Mission success increased from 27/30 without oversight (90.0%; 95% Wilson confidence interval: 74.4–96.5%) to 29/30 with deterministic guards only (96.7%; 83.3–99.4%) and 30/30 with the full overseer (100%; 88.7–100.0%). Mean turns decreased from 27.3 to 23.5 and 21.7, respectively. Thus guards alone recovered two of the three failed missions and 3.8 of the full module’s 5.6-turn efficiency improvement; advisory steering recov-

Table 7: Runs affected by degenerate behaviour in the three oversight conditions ($n = 30$ per condition). Patterns can overlap within a run.

Pattern	Full	Guards only	Off
Exploit without verification	0/30	2/30	30/30
Repetitive same-tool loop	0/30	4/30	20/30
Infinite recon loop (no exploit)	0/30	2/30	7/30
No blue-team transition	0/30	1/30	3/30

ered the remaining MQTT failure and reduced mean turns by a further 1.8. Redundant-call rates remained close across arms (38.7% off, 37.8% guards only, and 37.1% full), indicating that governance primarily bounds the consequences of repetition rather than removing the model’s underlying tendency to repeat.

Table 7 provides the stronger behavioural signal. Every no-overseer run exhibited exploit-without-verification, while deterministic guards reduced this pattern to 2/30 and the full overseer eliminated it in the evaluated runs. Guards similarly reduced same-tool loops from 20/30 to 4/30, reconnaissance loops from 7/30 to 2/30, and missing blue-team transitions from 3/30 to 1/30. The sole guards-only mission failure occurred in MQTT topic discovery, where the agent stalled until timeout; advisory steering recovered this remaining multi-step sequencing failure. MQTT is therefore the clearest evaluated case where advisory steering adds value beyond deterministic enforcement.

6. Further Discussions

6.1. Key Insights

A feasibility threshold has been crossed in a controlled bare-metal OT setting. Prior autonomous LLM pentesting work (Deng et al., 2024; Happe and Cito, 2023; Xu et al., 2024) operates on Linux targets where the agent can spawn shells, enumerate filesystems, and leverage familiar POSIX semantics. Bare-metal MCU targets offer none of these affordances: there is no shell to land in, no filesystem to pivot through, and success must be measured by device crash or register write. Our results show that *protocol-specific primitive tooling* makes this setting tractable in the evaluated controlled testbed: by giving the agent low-level control over CoAP

option bytes and Modbus Application Protocol (MBAP) length fields, and requiring it to construct byte sequences that trigger parser failures, APIOT bridges part of the semantic gap that has kept bare-metal targets out of scope for autonomous agents. An all-but-one pattern, with 100% success in 6 of 7 (protocol $\times$ topology) conditions and a single failure condition at CoAP on three-tier topology (3/10, 30%), combined with first exploits consistently within 5 tool calls, demonstrates feasibility for intentionally vulnerable emulated firmware.

The named-exploit-tool ablation sharpens this interpretation. Named exploit tools show that the easy T1 vulnerabilities can be triggered quickly when the attack is packaged for the model, whereas primitive tools provide stronger evidence of protocol-level operation: explicit field construction and more reusable network-level mitigation under held-out payload variants. This distinction matters because mission success alone is not sufficient evidence of reasoning. The primitive interface is valuable not merely because it can complete the mission, but because it exposes the intermediate protocol choices needed to audit, reproduce, and stress-test the agent’s behaviour. This does not by itself establish transfer to hardened production firmware.

For defenders, the implication is narrower but still important: in environments with exposed protocol-edge weaknesses, these results suggest that LLM agents can reduce the amount of manual protocol expertise needed to execute an attack-remediation loop. This conclusion holds for the emulated target class evaluated here (intentionally vulnerable Zephyr firmware and software-equivalent simulators); transfer to production OT firmware with hardened parsers, authentication, safety interlocks, and real silicon remains open.

Oversight as runtime reliability support. The LLM agent reliability literature (Liu et al., 2024) has identified repetition, inconsistency, and long-horizon failure as endemic to current LLMs. Proposed remedies are predominantly prompt-engineering or agent-level retries. The Claim 2 ablation shows that deterministic guards produce most of APIOT’s reliability improvement on the T1 baseline: they recovered two of three no-overseer mission failures and sharply reduced all four measured degenerate patterns. Optional advisory steering provided an incremental benefit, recovering the remaining MQTT failure and reducing mean completion turns from 23.5 to 21.7.

A key insight is that in 100% of overseer-OFF runs, at least one exploit call was not immediately followed by crash verification, not because the

agent *could not* perform crash verification, but because without an external invariant enforcing the sequence, it would issue additional probes or transition between phases before confirming crash state. Deterministic guards reduced this pattern to 2/30 runs, while the full overseer eliminated it in the evaluated sample. The remaining difference between guards-only and full oversight is concentrated in recovery from multi-step MQTT sequencing and a modest 1.8-turn efficiency improvement, rather than broad changes in the main agent’s call distribution.

6.2. Limitations

The Cortex-M3 board MAC constraint limits concurrent real QEMU MCU instances to one; we used software simulators for multi-device configurations, which were validated for crash-semantic fidelity but did not exercise QEMU-specific edge cases. The firmware and simulator vulnerabilities are intentional known/seeded weaknesses, so the results do not demonstrate zero-day discovery. Our exploit set covers parser-overflow, command-topic, and credential-style attacks; post-exploitation persistence, privilege escalation, and general lateral-movement strategy are out of scope. The testbed network was isolated; real OT network noise profiles may differ from our Linux traffic-shaping models. Primary-condition results use MiniMax M2.5, while Section 5.3 evaluates cross-model robustness and shows model-specific divergence in the MQTT hard condition. The reported defensive actions are network-level mitigations, not firmware repairs, and production deployment would require safety review and change-control processes. Both the tool-abstraction and three-arm governance ablations are limited to direct T1 baselines and therefore do not establish that their observed effects transfer to multi-hop conditions. Open questions remain on controlled quantification of cross-session memory benefits, authenticated/secured protocol variants, and transfer from QEMU-emulated firmware to physical Cortex-M3 silicon.

6.3. Ethics, Isolation, and Artifact Release

All experiments were performed against an isolated virtual lab owned by the authors. The agent was permitted to query lab state and send traffic only to the configured lab subnets, and its prompt explicitly forbade starting, stopping, respawning, or resetting lab devices. No live third-party systems, public Internet hosts, or production OT devices were targeted. The vulnerabilities evaluated here are intentionally seeded or known protocol-edge behaviours, so the work does not disclose new vendor zero-days.

7. Conclusion

We presented APIOT, an LLM agent framework that brings bare-metal industrial firmware—a target class that published autonomous agents, evaluated on Linux and web systems, do not reach—within the scope of autonomous vulnerability management. The support for this claim is empirical: 90.0% mission success across the primary discovery → exploitation → network-level mitigation → verification matrix and 85.3% over five frontier LLMs. A three-arm ablation showed that deterministic guards explain most of the runtime-governance benefit in the evaluated T1 conditions: success increased from 27/30 without oversight to 29/30 with guards only and 30/30 with guards plus advisory steering, while mean turns decreased from 27.3 to 23.5 and 21.7. The current deployment boundary—CoAP on three-tier Edge-Fog-Cloud routing—identifies multi-hop planning under stateless protocols as the next capability frontier.

Future work will pursue three directions: extending the evaluation to authenticated and encrypted protocol variants (e.g., CoAP over DTLS and Modbus security gateways); improving multi-hop lateral-movement planning under stateless protocols, which our topology results identify as the current capability boundary; and validating transfer from emulated firmware to physical MCU hardware. Longer term, we plan to quantify the benefit of cross-session memory and to assess agent-derived mitigations against production change-control requirements. Both APIOT and IoT Virtual Lab are released as open source to support reproducibility and further research.¹

CRedit authorship contribution statement

Adel ElZemity: Conceptualisation, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualisation, Writing – original draft, Writing – review and editing, Project administration, Resources. Budi Arief: Conceptualisation, Writing – review and editing, Supervision, Funding acquisition, Project administration. Shujun Li: Conceptualisation, Project administration, Writing – review and editing, Supervision, Funding acquisition. George Oikonomou: Writing – review and editing, Funding acquisition. James Pope: Writing – review and editing, Funding acquisition.

¹APIOT: <https://github.com/Adelsamir01/apiot>; IoT Virtual Lab: https://github.com/Adelsamir01/iot_vlab.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was partly supported by the Engineering and Physical Sciences Research Council (EPSRC), part of UK Research and Innovation (UKRI), under grant numbers EP/X036871/1 and EP/X036707/1.

Data availability

The artifact is available at <https://github.com/Adelsamir01/apiot>. It contains the APIOT code, IoT Virtual Lab configuration, prompt definitions, JSON tool schemas, analysis scripts, and sanitised run summaries. API keys, local host credentials, and environment-specific secrets are not released.

Appendix A. System Implementation Details

Appendix A.1. Prompt, Tool, and Artifact Transparency

The artifact package maps each methodological claim to a concrete source artifact. The guided prompt is `SYSTEM_PROMPT` in `apiot/core/agent.py`; the blind prompt is `BLIND_SYSTEM_PROMPT` in the same file. Blind mode is selected by the experiment runner, `scripts/run_experiment.py`, through `APIOT_BLIND_MODE`. Both prompt conditions pass the same `TOOL_SCHEMAS`, defined in `apiot/core/tools/registry.py`, to the OpenRouter-compatible chat-completion API. Context compaction is implemented in `apiot/core/compaction.py`, and runtime governance is implemented in `apiot/core/oversight.py`. These files expose the prompt text, tool names, descriptions, parameters, required fields, compaction trigger, preserved state, guard thresholds, and advisory-overseer snapshot used in the reported runs. In the public GitHub repository, paths of the form `apiot/<file>` refer to `<file>` at the repository root (the repository itself is the `apiot` package); plain-text prompt and schema copies are also provided under `artifact/`.

The guided prompt includes three protocol-specific hint families: (i) CoAP option-byte reasoning, including the 0xDD extended delta/length pattern; (ii)

Modbus MBAP length-field reasoning, including oversized `claimed_length` examples such as 2048; and (iii) MQTT topic-chain reasoning, including wildcard subscription, command-topic discovery, command publish, and telemetry-silence verification. The blind prompt keeps the same mission lifecycle, safety constraints, termination tokens, verification requirements, and tool roster, but removes those protocol-specific exploit examples. Thus blind failures are attributable to missing protocol hints rather than to a reduced action space.

The retained historical logs do not contain a normalised compaction-event table for every run. We therefore report token-use estimates for runs with positive token logs and describe the compaction mechanism directly from source, rather than claiming a compaction frequency distribution.

Appendix A.2. APIOT Implementation Details

APIOT implements a continuous THOUGHT $\rightarrow$ ACTION $\rightarrow$ OBSERVATION loop using an OpenAI-compatible client pointed at any OpenRouter-hosted LLM. The agent maintains a growing conversation history and submits 21 tool schemas defined as JSON Schema objects to the LLM at each turn, routing the returned calls through an Action Execution Layer that maps them to Python implementations.

Two-phase tool roster. Reconnaissance tools available in both phases include `get_network_state`, `get_actionable_targets`, `stealth_check`, and `inspect_lab`. Phase 1 (Red) tools include `coap_send`, `modbus_request`, `tcp_send`, `udp_send`, `mqtt_publish`, `mqtt_subscribe`, `verify_crash`, and `verify_shell`. Phase 2 (Blue) tools include `iptables_rule`, `modbus_fc_filter`, `coap_rate_limit`, `verify_patch`, `list_patches`, and `protocol_block`. Both phases are mandatory: a mission ends only at `TASK_COMPLETE` (all findings mitigated and verified) or `TASK_ABORTED` (unrecoverable infrastructure failure). Linux targets are additionally reachable via `run_command` (for banner grabbing, credential testing, and post-exploitation), `remote_exec` (SSH command execution on compromised hosts), and `create_tool` (runtime Python tool generation for novel attack scenarios not covered by primitives).

Context management. Token count is estimated at 4 characters per token. When utilisation reaches 70% of the model’s context window, `compaction.py` summarises older messages in-place and truncates tool results to 2 000 characters. The implementation keeps the system prompt and the latest eight messages, replacing the middle history with a structured summary. Separate experiment-level token-budget guards can abort long runs to prevent

runaway cost.

Appendix A.3. Overseer Mechanism Details

The Overseer implements seven mechanisms: (1) *Repetition guard*: blocks any `(tool, ip, args_hash)` tuple seen ≥ 3 times in a 20-call sliding window. (2) *Crash blocker*: prevents attacking a device already in "crashed" state. (3) *Mitigation blocker*: prevents re-attacking a vulnerability with a verified iptables/ACL rule on record. (4) *Stall detection*: injects a steering message at 8 turns without progress; forces phase transition or terminates at 15. (5) *Phase transition enforcement*: triggers the blue-team phase when all targets have been attempted; auto-signals completion when all findings are mitigated and verified. (6) *Strategy refresh*: every 5 turns, injects a refreshed attack plan. (7) *Overseer LLM*: an optional secondary model providing advisory support at stall events. The experiment runner exposes three modes: `off` bypasses all seven mechanisms, `guards-only` retains deterministic enforcement and heuristic fallbacks while disabling secondary-model calls, and `full` enables both guards and advisory steering.

The retained intervention summary from earlier analysis labels most entries only as `unknown`, with a smaller number of `strategy_refresh` entries. We therefore do not report per-intervention counts in the main results. A clean intervention-type table requires re-extracting typed overseer events from the full 30-run ablation logs.

Appendix A.4. IoT Virtual Lab Network and Device Details

IoT Virtual Lab boots real firmware images as QEMU (Bellard, 2005) processes, connects them via Linux TAP interfaces to a shared virtual bridge (`br0`, 192.168.100.0/24), and exposes a RESTful API for topology inspection. Three device classes are supported:

MIPS Linux routers (`qemu-system-mipsel`, Malta board): full Debian Linux with SSH and HTTP, representative of consumer router firmware.

ARM Linux gateways (`qemu-system-arm`, VersatilePB): ARM-based embedded Linux with multi-homed support, modelling edge gateway devices. A `br_internal` bridge (192.168.200.0/24) enables Purdue Model (PERA)-style segmentation.

ARM Cortex-M MCUs: Zephyr RTOS ELF images compiled with Kconfig overlays that strip IPv6 and shell to fit 64 KB RAM. The testbed supports two MCU targets: Cortex-M3 (`qemu-system-arm`, `lm3s6965evb`) and

Cortex-M4F (`qemu-system-arm`, mps2-an386). Three firmware variants provide protocol diversity: `zephyr_coap` (CoAP UDP:5683), `arm_modbus_sim` (Modbus/TCP:502), and `zephyr_echo` (echo TCP/UDP:4242). Since all reported exploits target the CoAP/Modbus application layer rather than CPU-specific behaviour, we do not claim separate core-specific exploitation results.

Network realism. `impair_network.sh` applies Linux `tc netem` rules to `br0` for configurable packet loss, latency, and jitter. `industrial_hmi_sim.py` generates background Modbus/CoAP traffic using a Poisson distribution, modelling SCADA polling noise.

Multi-device support and topology modes. The `lm3s6965evb` has a hardcoded MAC, limiting concurrent Cortex-M3 QEMU instances to one; lightweight Python simulators with identical crash/recovery semantics supplement QEMU for multi-MCU topologies. Four topology modes are supported: (T1) flat star, (T2) Purdue Model (PERA)-segmented (MCU behind multi-homed gateway on `br_internal`), (T3) Edge-Fog-Cloud three-tier, and a 15-node mesh. T2 and T3 require lateral movement through gateway devices to reach MCU targets.

References

- Alghamdi, T.A., Lasebae, A., Aiash, M., 2013. Security analysis of the Constrained Application Protocol in the Internet of Things, in: Proceedings of the 2nd International Conference on Future Generation Communication Technologies, IEEE. pp. 163–168. doi:10.1109/FGCT.2013.6767217.
- Anthropic, 2025. Disrupting the first reported AI-orchestrated cyber espionage campaign. Threat intelligence report. URL: <https://www.anthropic.com/news/disrupting-AI-espionage>.
- ARM Limited, 2010. Cortex-M3 Technical Reference Manual, Revision r2p0. ARM Limited. URL: <https://developer.arm.com/documentation/ddi0337/latest/>.
- Arvind, S., Narayanan, A., 2019. An overview of security in CoAP: Attack and analysis, in: Proceedings of the 2019 5th International Conference on Advanced Computing & Communication Systems, IEEE. pp. 655–660. doi:10.1109/ICACCS.2019.8728533.

- Banks, A., Gupta, R., 2014. MQTT version 3.1.1. OASIS Standard. URL: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
- Bellard, F., 2005. QEMU, a fast and portable dynamic translator, in: Proceedings of the 2005 USENIX Annual Technical Conference, USENIX Association. pp. 41–46. URL: <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator>.
- Bhamare, D., Zolanvari, M., Erbad, A., et al., 2020. Cybersecurity for industrial control systems: A survey. *Computers & Security* 89, 101677:1–101677:12. doi:10.1016/j.cose.2019.101677.
- Chen, D.D., Woo, M., Brumley, D., Egele, M., 2016. Towards automated dynamic analysis for Linux-based embedded firmware, in: Proceedings of the 2016 Network and Distributed System Security Symposium, Internet Society. doi:10.14722/ndss.2016.23415.
- Clements, A.A., Gustafson, E., Scharnowski, T., et al., 2020. HALucinator: Firmware re-hosting through abstraction layer emulation, in: Proceedings of the 29th USENIX Conference on Security Symposium, USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/clements>.
- Das, B.C., Amini, M.H., Wu, Y., 2025. Security and Privacy Challenges of Large Language Models: A Survey. *ACM Computing Surveys* 57. URL: <https://doi.org/10.1145/3712001>, doi:10.1145/3712001.
- Deng, G., Liu, Y., Mayoral-Vilches, V., et al., 2024. PentestGPT: Evaluating and harnessing large language models for automated penetration testing, in: Proceedings of the 33rd USENIX Security Symposium, USENIX Association. pp. 847–864. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/deng>.
- Fang, R., Bindu, R., Gupta, A., Kang, D., 2024. LLM agents can autonomously exploit one-day vulnerabilities. arXiv:2404.08144 [cs.CR]. doi:10.48550/arXiv.2404.08144.

- Feng, B., Mera, A., Lu, L., 2020. P2IM: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling, in: Proceedings of the 29th USENIX Security Symposium, USENIX Association. pp. 1237–1254. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/feng>.
- Ghazanfar, S., Hussain, F.B., ur Rehman, A., Fayyaz, U.U., Shahzad, F., Shah, G.A., 2020. IoT-Flock: An open-source framework for IoT traffic generation. Proceedings of the 2020 International Conference on Emerging Trends in Smart Technologies doi:10.1109/ICETST49965.2020.9080732.
- Guru, K., Moix, A., Klein, J., 2026. Mapping AI-enabled cyber threats. Anthropic research report. URL: <https://www.anthropic.com/research/attack-navigator>.
- Happe, A., Cito, J., 2023. Getting pwn'd by AI: Penetration testing with large language models, in: Proceedings of the 2023 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ACM. pp. 2082–2086. doi:10.1145/3611643.3613083.
- Happe, A., Kaplan, A., Cito, J., 2025. LLMs as hackers: Autonomous Linux privilege escalation attacks. Empirical Software Engineering 31, 70:1–70:50. doi:10.1007/s10664-025-10758-3.
- Kim, A., Kim, D., Kim, E., Kim, S., Jang, Y., Kim, Y., 2020. FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis, in: Proceedings of the 2020 Annual Computer Security Applications Conference, ACM. pp. 733–744. doi:10.1145/3427228.3427294.
- Lai, Y., Gao, H., Liu, J., 2020. Vulnerability mining method for the Modbus TCP using an anti-sample fuzzer. Sensors 20, 2040:1–2040:20. doi:10.3390/s20072040.
- Li, M.Q., Fung, B.C.M., 2025. Security concerns for large language models: A survey. Journal of Information Security and Applications 95, 104284. doi:<https://doi.org/10.1016/j.jisa.2025.104284>.
- Liu, X., et al., 2024. AgentBench: Evaluating LLMs as agents, in: Proceedings of the 2024 International Conference on Learning Representations.

- Modbus Organization, 2012. Modbus application protocol specification v1.1b3. Protocol specification. URL: https://modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf.
- Mohan, N., Kangasharju, J., 2016. Edge-Fog Cloud: A distributed cloud for internet of things computations, in: Proceedings of the 2016 Congress on Cloudification of the Internet of Things, IEEE. doi:10.1109/CIOT.2016.7872914.
- Mou, Y., Xue, Z., Li, L., Liu, P., Zhang, S., Ye, W., Shao, J., 2026. ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback. arXiv:2601.10156 [cs.CL]. doi:10.48550/arXiv.2601.10156.
- Oikonomou, G., Duquennoy, S., Elsts, A., Eriksson, J., Tanaka, Y., Tsiftes, N., 2022. The Contiki-NG open source operating system for next generation IoT devices. SoftwareX 18, 101089. doi:10.1016/j.softx.2022.101089.
- OpenAI, 2024. OpenAI o1 system card. arXiv:2412.16720 [cs.AI]. doi:10.48550/arXiv.2412.16720.
- OpenAI, 2025. Disrupting malicious uses of AI: October 2025. Threat intelligence report. URL: <https://openai.com/global-affairs/disrupting-malicious-uses-of-ai-october-2025/>.
- Raptis, E.K., Kapoutsis, A.C., Kosmatopoulos, E.B., 2025. Agentic LLM-based robotic systems for real-world applications: a review on their agentiveness and ethics. Frontiers in Robotics and AI 12. doi:10.3389/frobt.2025.1605405.
- Real Time Engineers Ltd, 2023. FreeRTOS - market leading RTOS for embedded systems. Website. URL: <https://www.freertos.org/>. accessed: 2025-01-15.
- Scarfone, K., Souppaya, M., Cody, A., Orebaugh, A., 2008. Technical Guide to Information Security Testing and Assessment. NIST Special Publication 800-115. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-115.

- Scharnowski, T., Bars, N., Schloegel, M., Gustafson, E., Muench, M., Vigna, G., Kruegel, C., Holz, T., Abbasi, A., 2022. Fuzzware: Using precise MMIO modeling for effective firmware fuzzing, in: Proceedings of the 31st USENIX Security Symposium, USENIX Association. pp. 1239–1256. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/scharnowski>.
- Shelby, Z., Hartke, K., Bormann, C., 2014. The Constrained Application Protocol (CoAP). RFC 7252. IETF. doi:10.17487/RFC7252.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems 36.
- Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., Thompson, M., 2023. Guide to Operational Technology (OT) Security. NIST Special Publication 800-82 Rev. 3. National Institute of Standards and Technology. Gaithersburg, MD. doi:10.6028/NIST.SP.800-82r3.
- team, B.S., 2024. From naptime to big sleep: Using large language models to catch vulnerabilities in real-world code. Blog post. URL: <https://projectzero.google/2024/10/from-naptime-to-big-sleep.html>.
- Van Buggenhout, E., 2024. Purple teaming: A comprehensive and collaborative approach to cyber security. Cyber Security: A Peer-Reviewed Journal 7, 207–216. doi:10.69554/WGLN8212.
- Walker, K., 2025. A summer of security: Empowering cyber defenders with AI. Blog post. URL: <https://blog.google/innovation-and-ai/technology/safety-security/cybersecurity-updates-summer-2025/>.
- Wan, S., Nikolaidis, C., Song, D., Molnar, D., et al., 2024. CyberSecEval 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models. arXiv:2408.01605 [cs.CR]. doi:10.48550/arXiv.2408.01605.
- Wang, H., Poskitt, C.M., Sun, J., 2025. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. arXiv:2503.18666 [cs.AI]. doi:10.48550/arXiv.2503.18666. accepted to the 48th IEEE/ACM International Conference on Software Engineering (ICSE 2026).

- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D., 2023. Self-consistency improves chain of thought reasoning in language models, in: Proceedings of the 2023 International Conference on Learning Representations.
- Williams, T.J., 1994. The purdue enterprise reference architecture. *Computers in Industry* 24, 141–158. doi:10.1016/0166-3615(94)90017-5.
- Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., Song, D., Li, B., 2025. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning, in: Proceedings of the 42nd International Conference on Machine Learning.
- Xu, J., Stokes, J.W., McDonald, G., Bai, X., Marshall, D., Wang, S., Swaminathan, A., Li, Z., 2024. AutoAttacker: A large language model guided system to implement automatic cyber-attacks. *arXiv:2403.01038 [cs.CR]*. doi:10.48550/arXiv.2403.01038.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2023. ReAct: Synergizing reasoning and acting in language models, in: Proceedings of the 2023 International Conference on Learning Representations. URL: https://openreview.net/forum?id=WE_vluYUL-X.
- Zephyr Project, 2024. Zephyr RTOS documentation, v3.7.0. Website. URL: <https://docs.zephyrproject.org/>.